



UNIVERSITY OF BUCHAREST

FACULTY OF
MATHEMATICS AND
COMPUTER SCIENCE



INFORMATION TECHNOLOGY SPECIALIZATION

Bachelor's Thesis

AUTOMATIC PUZZLE SOLVING USING COMPUTER VISION TECHNIQUES

Graduate

Moraru Ilinca-Ana

Scientific Coordinator

Associate Professor Dr. Bogdan Alexe

Bucharest, June 2025

Rezumat

Lucrarea abordează problema rezolvării unui puzzle de dimensiuni variabile (între 100 și 400 de piese) prin metode euristice de identificare a potrivirilor corecte folosind un algoritm iterativ, determinist, ce efectuează deciziile de alăturare sau desfacere. Cele mai bune rezultate au fost obținute prin folosirea euristicilor simple, bazate pe distanțe între caracteristici simple precum culori sau gradienti. Algoritmul de rezolvare a unui puzzle este alcătuit din trei etape, pornind de la alăturări simple de piese în grupuri compacte folosind euristici de potrivire simple, pentru ca, pe măsură ce piesele din puzzle sunt poziționate corect, locurile rămase să fie completate folosind euristici mai complexe. Avantajul acestei abordări, spre deosebire de paradigma populară a rețelelor neurale, este un control amănunțit cu privire la comportamentul programului cât și un timp de execuție mai mic.

Abstract

This paper explores the problem of solving jigsaw puzzles of varying sizes (between 100 and 400 pieces) using heuristic methods to find correct matches. The proposed approach uses a deterministic and iterative algorithm that performs joining or splitting decisions. The best results were achieved with simple heuristics based on distances between basic features like colors and gradients. The algorithm is divided into three stages. It starts by forming compact groups of pieces using basic matching rules, and as more pieces are placed correctly, the remaining spots are filled using complex heuristics. Compared the popular neural network-based methods, this approach offers more control over the algorithm's behavior and requires less computation time.

Contents

1	Introduction	4
2	Merging Puzzle Pieces Using Vision-Inspired Distance Metrics	7
3	The Greedy Puzzle Piece Merging Algorithm	15
3.1	Merging Groups	15
3.2	Algorithm Stages	18
3.2.1	Stage I: Merging Groups Based on Edge Pairs with Low Matching Distance	18
3.2.2	Stage II: The Voting System	20
3.2.3	Stage III: Merging the Remaining Pieces to the Main Group	21
3.3	Results	23
	Bibliography	28

Chapter 1

Introduction

The problem of solving jigsaw puzzles with a large number of pieces has significant relevance in the field of computer vision, particularly in the reconstruction of documents or murals. This type of problem has practical applications in various fields, such as archaeology, where researchers attempt to match stone fragments in order to reconstruct frescoes. An ongoing large-scale project, the RePAIR Project, aims to reassemble fragments of frescoes discovered in the city of Pompeii, such as those in Figure 1.1. There are over 10 thousand fragments belonging to different frescoes, ungrouped by the walls they originate from, making reassembly nearly impossible to achieve by human effort alone.



Figure 1.1: **RePAIR Project.** *Example of frescoes reconstructed by archaeologists.*

This paper presents a solution to the problem of automatically solving square-piece jigsaw puzzles in which the orientation of each piece in the final solution is unknown. We focus on puzzles with sizes ranging from 10×10 to 20×20 , corresponding to 100 to 400 individual pieces. The input consists of the target puzzle dimensions and a set of corresponding pieces, each randomly rotated by an angle $\theta \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$. These pieces form an unordered set, with no information about their original position or orientation, as illustrated in Figure 1.2. Solving the puzzle involves determining, for each piece P_i , the triplet (x_i, y_i, θ_i) , which specifies its position and rotation in the final reconstructed image.

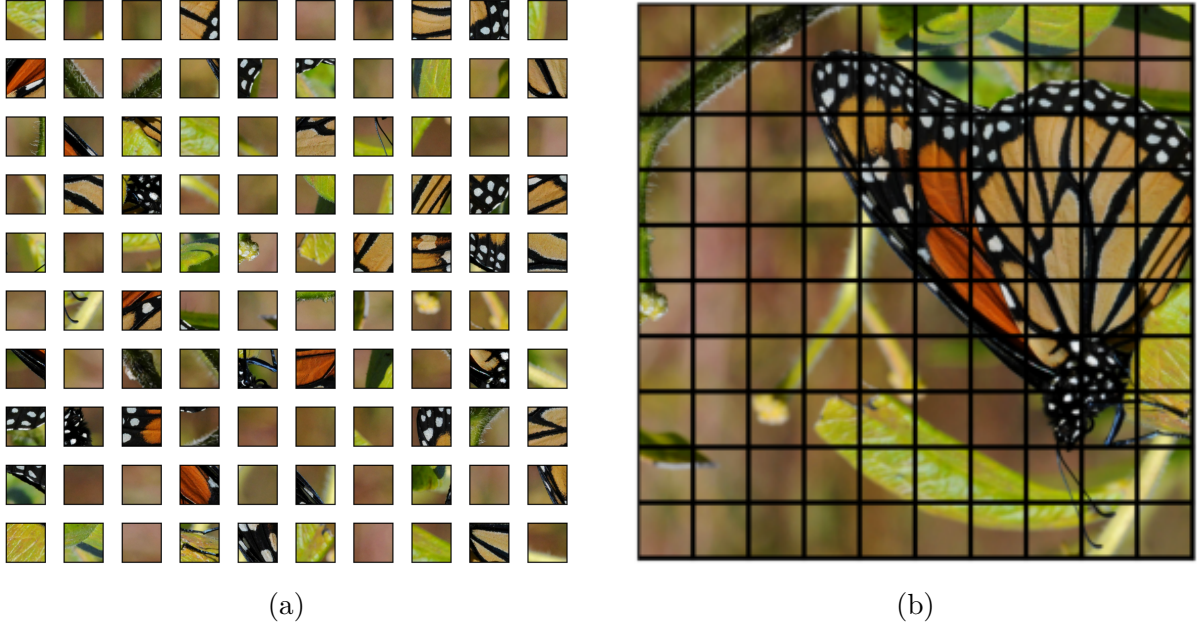


Figure 1.2: **Example of a puzzle with 10×10 pieces.** (a) *Initial pieces, in random order and orientation.* (b) *Solved puzzle.*

The main contribution of this paper is the proposal of a sequential algorithm for solving jigsaw puzzles, where the merging of pieces is guided by various matching distances based on color differences, inspired by computer vision techniques. The proposed solution performs piece matching at the group level: initially, each piece is treated as an individual group, and as the algorithm progresses, these groups grow by merging compatible pieces.

The algorithm consists of the following three stages:

- i) Identifying groups of pieces that are adjacent in the ground truth solution by merging pieces whose edges have a matching distance below a predefined threshold;
- ii) Merging groups using a voting-based strategy;
- iii) Fitting the unmatched pieces by disassembling the remaining small groups and merging them into the main group.

The problem of automatic puzzle solving has been extensively studied, with a vast body of literature addressing various approaches [1, 2, 3, 4, 5, 6]. In studies that focus on square-piece puzzles with unknown rotations using iterative algorithms, the proposed methods are generally Greedy in nature. Examples include local minimum identification (the Hungarian Method) and the simultaneous search for all four neighbors of a piece [1], the construction of subtrees under geometric constraints [3], and the use of Loop Constraints [6].

A key aspect of solving jigsaw puzzles lies in defining an effective matching metric to evaluate the compatibility between two edges. Notable approaches include combining

color-based similarity scores with Gist descriptors [4], or integrating color differences with global puzzle statistics [2]. In their evaluation of five different matching metrics, Cho et al. [2] showed that simple color difference between adjacent edges yielded the highest discriminative power. Other notable methods include the use of Mahalanobis distance [3, 5].

Chapter 2

Merging Puzzle Pieces Using Vision-Inspired Distance Metrics

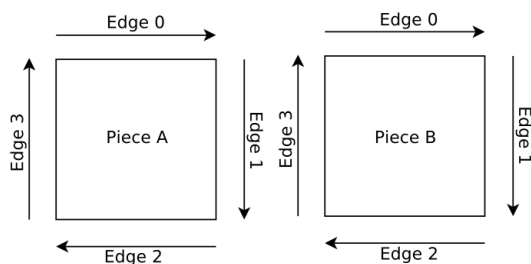


Figure 2.1: **Edges Encoding.**

A crucial step in solving a jigsaw puzzle is defining a metric that quantifies the compatibility between pieces. Such a metric should behave like a distance function: (i) it should return values close to 0 for pairs of optimally rotated edges that are adjacent in the ground truth solution; (ii) it should return high values, close to 1 , for pairs of edges that do not correspond to adjacent and optimally rotated pieces in the ground truth solution.

Since solving this problem involves determining the optimal rotation of each puzzle piece, every edge is stored in memory according to a clockwise border traversal, as illustrated in Figure 2.1. To quantify the similarity between two edges, we use distance functions that operate on the corresponding pixels of the two edges, taking into account the following important aspect: the pixels of one edge are considered in the order they are stored in memory, while the pixels of the second edge are considered in reverse order. This simulates the way in which the two edges would physically align in an actual puzzle.

To evaluate the compatibility of two pieces, we compute the distance between all possible pairs of corresponding edges, resulting in a total of 16 distance values. We consider the following pairs of edges:

$$\begin{aligned}
(e_A, e_B) \in \{ & (e_0^A, e_0^B), (e_0^A, e_1^B), (e_0^A, e_2^B), (e_0^A, e_3^B), \\
& (e_1^A, e_0^B), (e_1^A, e_1^B), (e_1^A, e_2^B), (e_1^A, e_3^B), \\
& (e_2^A, e_0^B), (e_2^A, e_1^B), (e_2^A, e_2^B), (e_2^A, e_3^B), \\
& (e_3^A, e_0^B), (e_3^A, e_1^B), (e_3^A, e_2^B), (e_3^A, e_3^B) \}
\end{aligned}$$

The computation of these distances is performed once, at the beginning of the algorithm. Subsequently, depending on how the pieces are merged, the rotation variable of each piece, $\theta \in \{0, 1, 2, 3\}$, is updated accordingly.

In what follows, edge pairs of the form (e_A, e_B) , of length n , refer to pairs of edges belonging to different pieces, and are represented as:

$$e_X = (p_X^1, p_X^2, \dots, p_X^n),$$

where p_X^i represents a characteristic of the i -th pixel of edge X , typically considered to be the RGB triplet:

$$p_X^i = (p_{X_R}^i, p_{X_G}^i, p_{X_B}^i).$$

In the following section, we present the different distance functions used by the developed algorithm to assess piece compatibility.

Color-based edge distance. We measure the color variation between two edges e_A and e_B of two puzzle pieces by computing the RGB color differences between the pixels p_A^i and p_B^i along the two edges. Equation 2.1 defines this distance:

$$d_{\text{color}}(e_A, e_B) = \frac{1}{n} \sum_{i=1}^n \sigma \left(\frac{4 \|p_A^i - p_B^{n-i+1}\|_2 - 2}{2} + 0.5 \right), \quad \text{where } \sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.1)$$

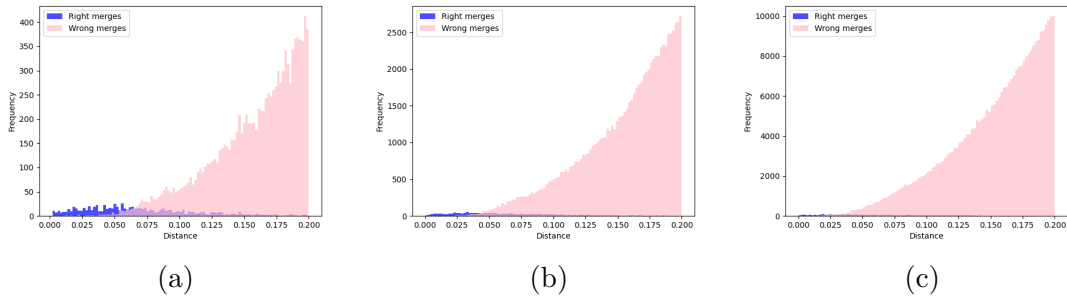


Figure 2.2: **Distribution of *color-based edge distances* for adjacent (blue) and non-adjacent (pink) edge pairs in the ground truth solution, across 10 puzzles.** (a) *Puzzles with 10×10 pieces.* (b) *Puzzles with 15×15 pieces.* (c) *Puzzles with 20×20 pieces.*

Although for most edge pairs that correspond to adjacent pieces in the ground truth solution the color-based distance values tend to be close to 0, separating adjacent edge pairs and non-adjacent edge pairs is difficult, as shown in Figure 2.2. The results are slightly better when using RGB channels compared to using a grayscale conversion of the pixel norms.

Gradient-based edge distance. This distance measures the difference in gradient magnitudes between corresponding pixels i on the edges e_A and e_B , computed by subtracting $\|\nabla G_{e_A}^i\|_2$ and $\|\nabla G_{e_B}^{n-i+1}\|_2$. The metric is defined in Equation 2.2. **Gradients** are computed by applying a Gaussian filter of size 5×5 to the original image, followed by two Sobel filters of size 5×5 that capture color variation along the x and y axes. As a result, there are six gradient vectors per edge: one for each of the RGB color channels in both the x and y directions. **Gradient magnitudes** for each pixel i , denoted as $\|\nabla G^i\|_2$, are computed as shown in Equation 2.3.

$$d_{\text{gradient } e_A e_B} = \frac{1}{n} \sum_{i=1}^n \left[\sigma \left(\frac{4 \left| \|\nabla G_{e_A}^i\|_2 - \|\nabla G_{e_B}^{n-i+1}\|_2 \right| - 2}{2} + 0.5 \right) \right] \quad (2.2)$$

$$\begin{aligned} \|\nabla G^i\|_2 &= \sqrt{G_x^i{}^2 + G_y^i{}^2}, \quad \text{where } \nabla G = (G_x, G_y) \\ G_x^i &= \sqrt{G_{x,R}^i{}^2 + G_{x,G}^i{}^2 + G_{x,B}^i{}^2}, \quad G_y^i = \sqrt{G_{y,R}^i{}^2 + G_{y,G}^i{}^2 + G_{y,B}^i{}^2} \end{aligned} \quad (2.3)$$

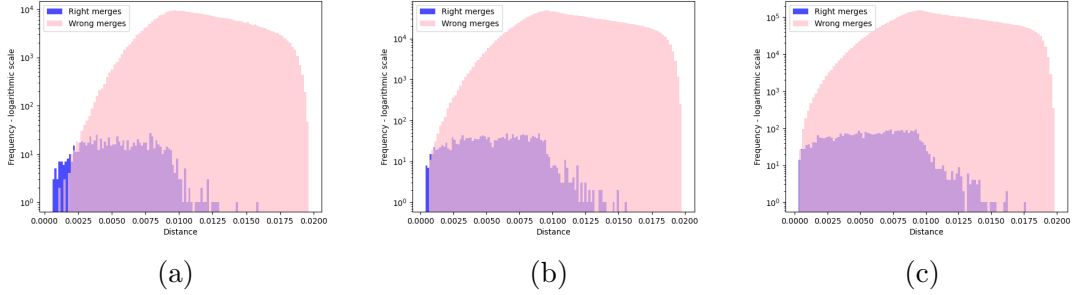


Figure 2.3: **Distribution of *gradient-based edge distances* for adjacent (blue) and non-adjacent (pink) edge pairs in the ground truth solution across 10 puzzles, logarithmic scale.** (a) *Puzzles with 10×10 pieces.* (b) *Puzzles with 15×15 pieces.* (c) *Puzzles with 20×20 pieces.*

Although intuitively the *gradient-based distance* would seem to offer more complex information, its discriminative power proves to be lower than that of the *color-based distance*. As shown in Figure 2.3, only a few isolated cases near the value 0 indicate gradient distances for adjacent edge pairs that do not overlap with those of non-adjacent pairs in the ground truth. The actual color context between pixels p_A^i and p_B^i is missing, and the magnitude of color variation within the two pieces can coincide even when the actual

RGB values differ significantly.

Gradient sum of two edges. We quantify the total level of detail present in the edges e_A and e_B by summing the gradient magnitudes $\|\nabla G_{e_A}^i\|_2$ and $\|\nabla G_{e_B}^i\|_2$ for each pixel i , as defined in Equation 2.4:

$$SG(e_A, e_B) = \sum_{i=1}^n \left[\sigma\left(\frac{4\|\nabla G_{e_A}^i\|_2 - 2}{2} + 0.5\right) + \sigma\left(\frac{4\|\nabla G_{e_B}^{n-i+1}\|_2 - 2}{2} + 0.5\right) \right] \quad (2.4)$$

This metric alone cannot classify edge pairs as adjacent or non-adjacent in the final solution. However, when combined with the *color-based edge distance*, it contributes to an improvement of the overall performance.

Normalized color-based edge distance. This distance is computed by dividing the *color-based edge distance* between two edges e_A and e_B by their corresponding *gradient sum* $SG(e_A, e_B)$, as defined in Equation 2.5. The *normalized color distance* tends to be smaller when the edges contain more visual detail, drawing inspiration from the human strategy of puzzle solving, where pieces with rich detail are typically merged first.

$$d_{\text{color}}^{\text{normalized}}(e_A, e_B) = \frac{d_{\text{color}}(e_A, e_B)}{SG(e_A, e_B)} \quad (2.5)$$

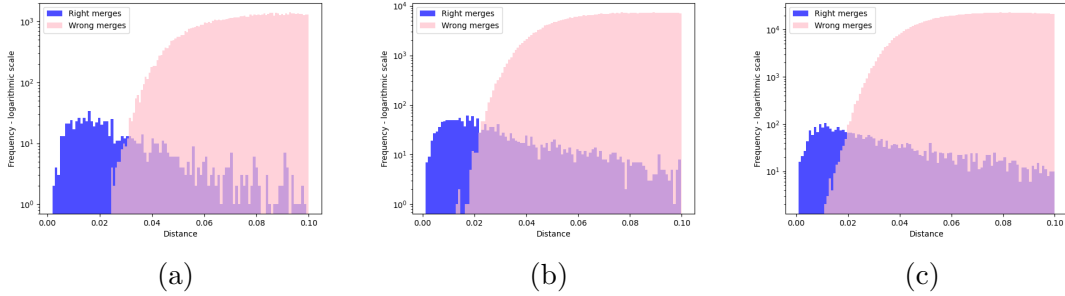


Figure 2.4: **Distribution of *normalized color-based distances* for adjacent (blue) and non-adjacent (pink) edge pairs in the ground truth solution across 10 puzzles, logarithmic scale.** (a) *Puzzles with 10×10 pieces.* (b) *Puzzles with 15×15 pieces.* (c) *Puzzles with 20×20 pieces.*

This distance can be effectively used during the first stage of the algorithm, as its ability to distinguish between matching and non-matching edge pairs is improved compared to the previously proposed distances, as shown in Figure 2.4. Edge pairs that are relatively smooth, with few gradients but similar colors, are discouraged, while pairs with both low color difference and rich detail are preferred. The likelihood of error remains low by using a per-edge-pair threshold of 0.08 and an average of 0.02 across all merges

between puzzle piece groups. For puzzles up to size 14×14 , the separation between matching and non-matching edge pairs is significantly improved, with fewer than 10 unmatched pieces remaining outside the main group. However, for more complex puzzles, the *normalized color distance* alone is not sufficient and a few matching errors still occur.

Improved normalized color-based edge distance. This distance incorporates a penalty based on the number k of pixels i along the edges e_A and e_B for which the individual pixel-level color distance $d_{\text{color}}^i(e_A, e_B)$ exceeds a threshold of 0.3 , this value being considered a significant color difference. The *improved normalized color distance* increases exponentially with respect to the *normalized color distance* for high values of k . The metric is defined in Equation 2.6:

$$d_{\text{color}}^{\text{in}}(e_A, e_B) = d_{\text{color}}^{\text{normalized}}(e_A, e_B) \cdot 5^k, \quad (2.6)$$

where $k = \text{Card} \{i \in \{1, \dots, n\} \mid d_{\text{color}}^i(e_A, e_B) > 0.3\}$

and $d_{\text{color}}^i(e_A, e_B) = \sigma \left(\frac{4 \|p_A^i - p_B^{n-i+1}\|_2 - 2}{2} + 0.5 \right)$

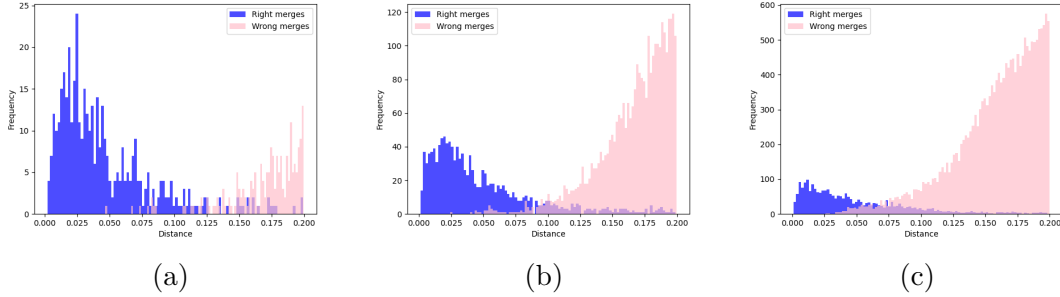


Figure 2.5: **Distribution of *improved normalized color distances* for adjacent (blue) and non-adjacent (pink) edge pairs in the ground truth solution across 10 puzzles.** (a) *Puzzles with 10×10 pieces.* (b) *Puzzles with 15×15 pieces.* (c) *Puzzles with 20×20 pieces.*

For an edge pair whose corresponding pieces are adjacent in the ground truth solution, k tends to be small, typically capturing differences around object boundaries in the image. At the same time, significant differences exceeding the 0.3 threshold disqualify many edge pairs whose pieces are not adjacent in the final solution. This metric serves as a first step in identifying matching edge pairs in large puzzles, where the basic *normalized color distance* may lead to errors. Compared to the *normalized color distance*, more correct edge pairs fall below the matching threshold, as illustrated in Figure 2.5, although imprecisions still occur on both edges of the threshold.

Reciprocal color-based edge distance. Building on the concept of the "*Best Buddy Score*" used in [3], a new metric is proposed, called the *reciprocal color-based distance*. This metric quantifies the mutual compatibility of two edges with a small color distance, relative to the smallest distances that each edge has with any other. For each of the two edges e_A and e_B , the edges e_C and e_D are determined such that $d_{\text{color}}^{\text{normalized}}(e_A, e_C)$ and $d_{\text{color}}^{\text{normalized}}(e_B, e_D)$ are minimized. The *normalized color distance* between e_A and e_B is then divided by these minimum values, as defined in Equation 2.7:

$$d_{\text{color}}^{\text{reciprocal}}(e_A, e_B) = \frac{d_{\text{color}}^{\text{normalized}}(e_A, e_B)}{\min_{e_C \in \text{edges}} d_{\text{color}}^{\text{normalized}}(e_A, e_C)} + \frac{d_{\text{color}}^{\text{normalized}}(e_A, e_B)}{\min_{e_D \in \text{edges}} d_{\text{color}}^{\text{normalized}}(e_B, e_D)} - 1 \quad (2.7)$$

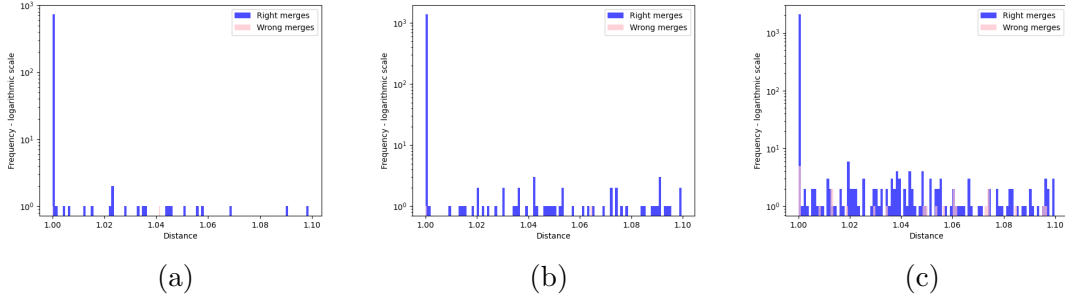


Figure 2.6: **Distribution of *reciprocal color-based distances* for adjacent (blue) and non-adjacent (pink) edge pairs in the ground truth solution across 10 puzzles, logarithmic scale.** (a) *Puzzles with 10×10 pieces.* (b) *Puzzles with 15×15 pieces.* (c) *Puzzles with 20×20 pieces.*

If the best match e_C for edge e_A is in fact e_B , and the best match e_D for edge e_B is e_A , the two edges e_A and e_B exhibit strong reciprocity, and their *reciprocal color-based distance* equals 1 - the minimum possible value of this metric. Otherwise, the lower the values of $d_{\text{color}}^{\text{normalized}}(e_A, e_C)$ and $d_{\text{color}}^{\text{normalized}}(e_B, e_D)$, the higher the *reciprocal distance* of the pair (e_A, e_B) becomes. It is possible to find edge pairs (e_A, e_B) such that $e_C = e_B$ while $e_D \neq e_A$. In such situations, the *reciprocal color-based distance* highlights potential mismatches not captured by the *normalized color-based distance* alone.

Figure 2.7a illustrates all *normalized color distances* between a given edge e_X and all other edges in the puzzle (left), and the corresponding *reciprocal color distances* (right), with the ground truth matching pair (e_X, e_Y) highlighted in blue. Although there are 12 edge pairs with lower normalized distances than (e_X, e_Y) , by considering the relatively high normalized distances associated with edge e_Y , the pair (e_X, e_Y) stands out as having the smallest *reciprocal color-based distance*. Figure 2.7b shows the distance values for an edge e_X located on the boundary of the final solution, meaning it is not adjacent to any

other piece. Even though some of the *normalized color-based distances* are small enough to be considered likely matches, they are relatively high when compared to the distribution of distances for those edges. Consequently, the *reciprocal color distances* for such pairs exceed 3, resulting in lower priority during puzzle reconstruction.

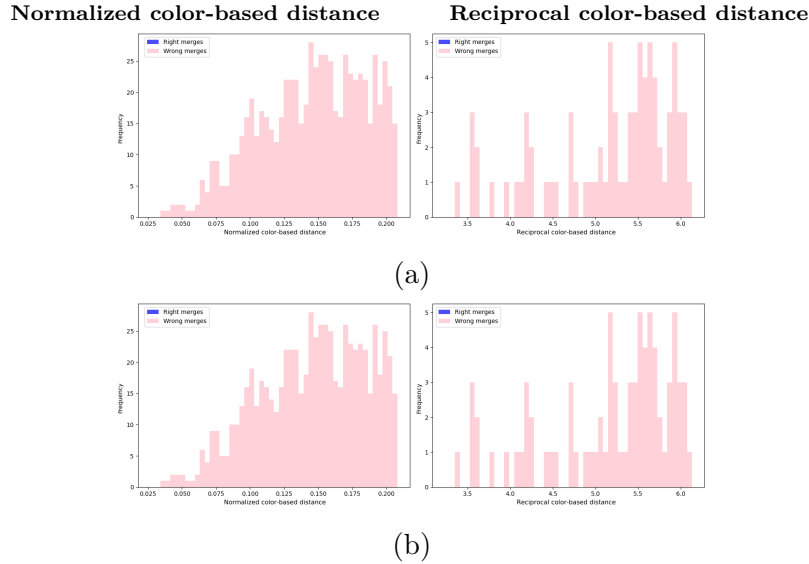


Figure 2.7: **Example of edge distance distributions.** (a) *Normalized and reciprocal color distances for an edge located inside the final solution.* (b) *Normalized and reciprocal color distances for an edge located on the border of the final solution.*

This metric builds upon the assumptions previously made by the *normalized color-based distance*, while also performing normalization relative to the pairs (e_A, e_C) and (e_B, e_D) . Its discriminative power is high, as shown in Figure 2.6. However, there are cases where both edges are located on the border of the final solution and should not be matched to any other edge, yet their *reciprocal color-based distance* is minimal, especially when those edges share similar colors and contain few visual details (e.g., clear sky regions). As such, the *reciprocal color-based distance* will not be used as a standalone metric, but rather as a complementary criterion, providing a broader context for match likelihood assessment.

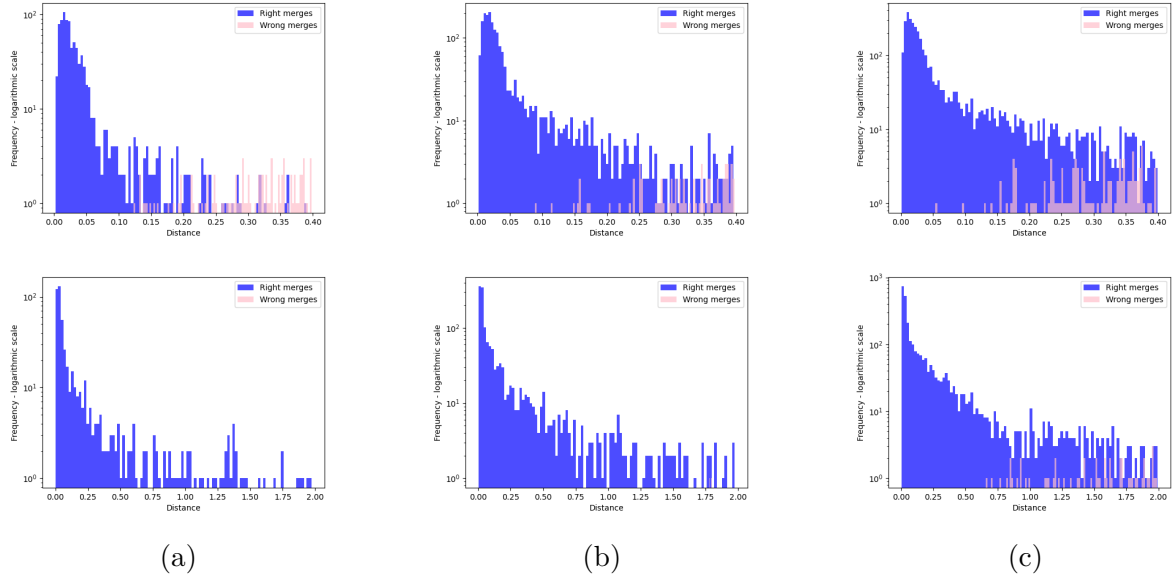


Figure 2.8: **Distribution of the *reciprocal color-based distance multiplied by other distance metrics* for adjacent (blue) and non-adjacent (pink) edge pairs in the final solution across 10 puzzles**, in logarithmic scale. (1) *Reciprocal color-based distance · normalized color-based distance*. (2) *Reciprocal color-based distance · improved normalized color-based distance*. (a) *Puzzles with 10×10 pieces*. (b) *Puzzles with 15×15 pieces*. (c) *Puzzles with 20×20 pieces*.

Multiplying the *reciprocal color-based distance* with either the *normalized color-based distance* or the *improved normalized color-based distance* enables better separation between edge pairs that are adjacent in the final solution and those that are not. The second product results in more correctly identified adjacent pairs, as shown in Figure 2.8. However, there are still some adjacent pairs in the final solution whose combined distances exceed the discriminative threshold. Therefore, these combined distances are best suited for an early stage of the solving process, where the goal is to confidently identify as many correct adjacent edge pairs as possible while tolerating the omission of a few harder-to-detect matches. These multiplicative combinations leverage the strengths of both individual distances, ultimately identifying more adjacent edge pairs in the final solution, and with higher precision than either metric used alone.

Chapter 3

The Greedy Puzzle Piece Merging Algorithm

3.1 Merging Groups

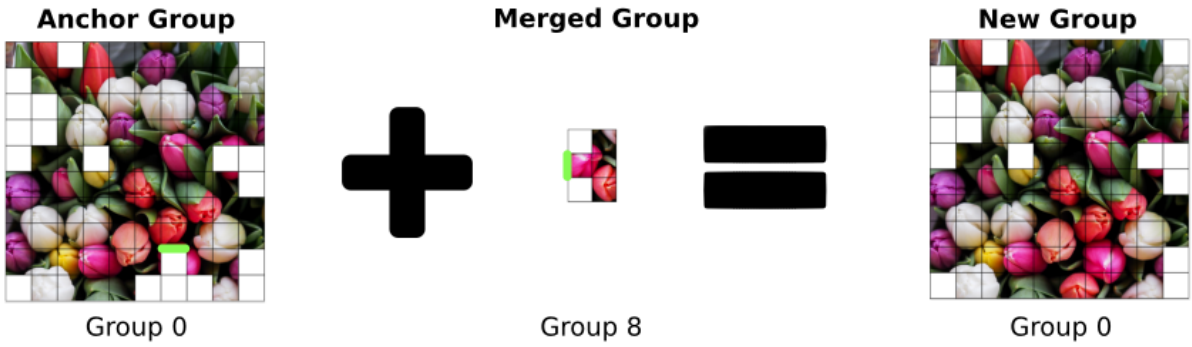


Figure 3.1: **Example of merging two groups G_a and G_m .**

In an $N \times N$ puzzle, each *group* consists of one or more *pieces* P_i , whose positions and rotations are described by the corresponding triplet:

$$P_i = (x_i, y_i, \theta_i), \quad \text{where } 0 \leq x_i, y_i < N, \quad \theta_i \in \mathbb{Z}_4.$$

Each edge E_i of a piece P_i is assigned an $id_i \in \mathbb{Z}_4$, uniquely associated with that edge. IDs are assigned to a piece's edges at the beginning of the algorithm based on a clockwise traversal: 0 – top, 1 – right, 2 – bottom, 3 – left. These IDs remain unchanged throughout the algorithm. Each edge E_i also has an associated orientation $orientation_i \in \mathbb{Z}_4$, which represents the result of adding the edge ID (id_i) to the rotation that piece P_i undergoes when it is merged into the group it belongs to. This expresses the "real" orientation of the edge, using the same encoding: 0 – top, 1 – right, 2 – bottom, 3 – left.

To merge two groups, we define:

- i) the *anchor group* G_a , in which all pieces $P_i \in G_a$ retain their current values (x_i, y_i, θ_i) and edge values $(id_i, orientation_i)$;
- ii) the *merged group* G_m , which will be transformed into G_m^* by applying the appropriate rotation and translation, such that:

$$\begin{aligned} P_i &= (x_i, y_i, \theta_i) \rightarrow P_i^* = (x_i^*, y_i^*, \theta_i^*), \\ E_j &= (id_j, orientation_j) \rightarrow E_j^* = (id_j, orientation_j^*), \quad \forall P_i \in G_m, \quad \forall E_j \in Edges(P_i). \end{aligned}$$

Two groups G_a and G_m can be merged only if the following conditions are satisfied:

$$\begin{aligned} (x_i, y_i) &\neq (x_j, y_j), \quad \forall P_i = (x_i, y_i, \theta_i) \in G_a, \quad \forall P_j = (x_j, y_j, \theta_j) \in G_m^* \\ 0 &\leq x_k, y_k < N, \quad \forall P_k = (x_k, y_k, \theta_k) \in G_a \cup G_m^* \end{aligned} \quad (3.1)$$

The merging of groups G_a and G_m is triggered by a low distance between two edges, denoted E_a and E_m , which belong to two distinct pieces $P_a \in G_a$ and $P_m \in G_m$, respectively. To verify the conditions for merging G_a and G_m , we compute a transformation triplet $(\nabla_x, \nabla_y, \nabla_\theta)$, which is applied to group G_m , resulting in a rotated and translated version G_m^* that simulates the merge.

The computation of the triplet $(\nabla_x, \nabla_y, \nabla_\theta)$ is described and illustrated below.

The orientations $orientation_a$ and $orientation_m$ of edges E_a and E_m , relative to their groups G_a and G_m , are calculated using the following formula:

$$orientation_{E_X} = id_{E_X} + \theta_{P_X} \quad (3.2)$$

Figure 3.1 shows an example of two groups being merged, based on a strong match between edges E_a and E_m , marked in green. These edges have IDs $id_a = 1$ and $id_m = 0$, and their corresponding pieces P_a and P_m have rotations $\theta_a = 1$ and $\theta_m = 3$. Applying Equation 3.2, we obtain $orientation_a = 2$ and $orientation_m = 3$, as illustrated in Figure 3.2a.

The pair $(orientation_a, orientation_m)$ corresponding to the edge pair E_a and E_m must, after alignment, match one of the following valid orientation pairs: (right, left), (left, right), (bottom, top), (top, bottom), encoded as $Valid_Orientations \in \{(1, 3), (3, 1), (2, 0), (0, 2)\}$. Therefore, $orientation_m^*$ will be adjusted according to $orientation_a$, so that $(orientation_a, orientation_m^*) \in Valid_Orientations$.

Next, we determine the rotation ∇_θ that group G_m must undergo in order to align with group G_a , according to Equation 3.3:

$$\nabla_\theta = orientation_m^* - orientation_m \quad (3.3)$$

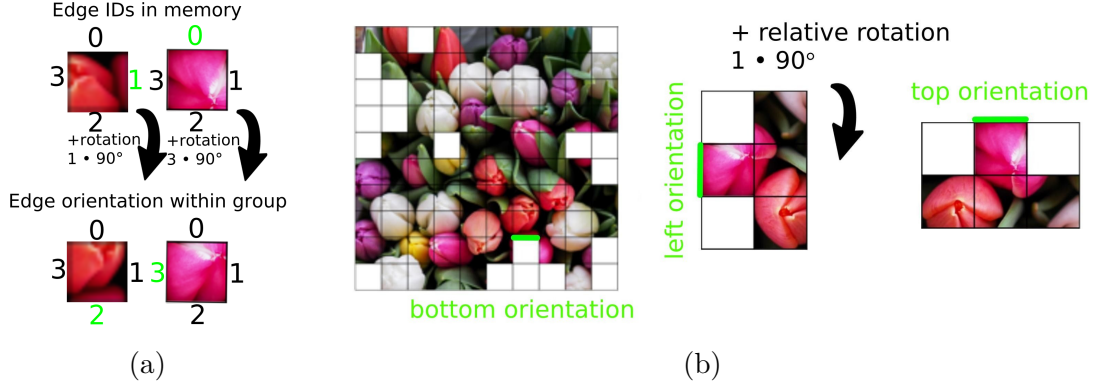


Figure 3.2: **Rotation of group G_m .** (a) Computation of the orientations $orientation_a$ and $orientation_m$ of edges E_a and E_m . (b) Rotation of group G_m based on the orientations $orientation_a$ and $orientation_m$ of Edges E_a and E_m .

Figure 3.2b illustrates the rotation ∇_θ that will be applied to group G_m . Given the value $orientation_a = 2$, $orientation_m^*$ becomes 0. By applying Equation 3.3, the resulting rotation for group G_m is $\nabla_\theta = 1$. Thus, group G_m is rotated by 90°.

The next step consists in determining the translations ∇_x and ∇_y along the Ox and Oy axes of group G_m . These are computed based on the coordinates x_a , y_a , x_m , y_m of the pieces P_a and P_m , as well as the *relative offset* $\delta = (\delta_x, \delta_y)$, which depends on the pair of orientations $(orientation_a, orientation_m^*)$. The *relative offset* δ is defined as follows:

$$(\delta_x, \delta_y) = \begin{cases} (1, 0), & \text{if } (orientation_a, orientation_m^*) = (1, 3) \\ (-1, 0), & \text{if } (orientation_a, orientation_m^*) = (3, 1) \\ (0, 1), & \text{if } (orientation_a, orientation_m^*) = (2, 0) \\ (0, -1), & \text{if } (orientation_a, orientation_m^*) = (0, 2) \end{cases} \quad (3.4)$$

The translations ∇_x and ∇_y are then calculated using Equation 3.5:

$$\begin{aligned} \nabla_x &= x_a + \delta_x - x_l \\ \nabla_y &= y_a + \delta_y - y_l \end{aligned} \quad (3.5)$$

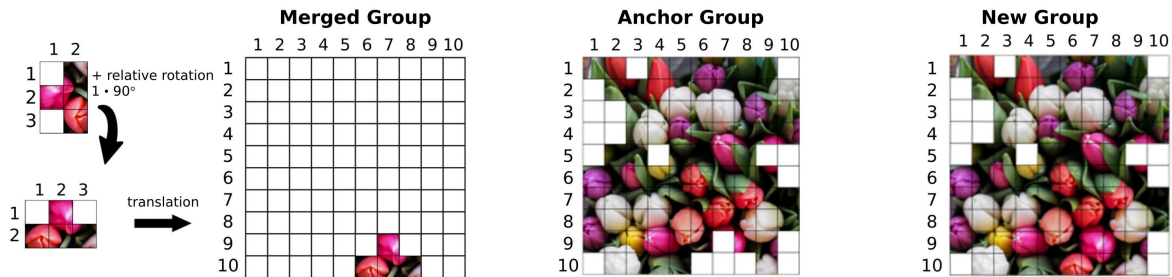


Figure 3.3: **Translation of the merged group and merging of the two groups.**

Figure 3.3 illustrates the translation applied to group G_m after the rotation step. Piece P_a has coordinates $x_a = 7$ and $y_a = 8$, while piece P_m has $x_m = 2$ and $y_m = 1$. Given that the pair of edge orientations is (bottom, top), according to Equation 3.4, we have $(\delta_x, \delta_y) = (0, 1)$. Applying Equation 3.5, we obtain $\nabla_x = 5$ and $\nabla_y = 8$. Thus, each piece P_i in group G_m will have its coordinates x_i, y_i incremented by $\nabla_x = 5$ and $\nabla_y = 8$. For instance, piece P_m , originally at $(2, 1)$, will be positioned at $(7, 9)$.

3.2 Algorithm Stages

The algorithm aims to find the optimal ordering of the performed merges, so that the puzzle is solved as close as possible to the ground truth solution. In order to achieve this ordering, various strategies and distance metrics presented in Chapter 2 are used. The algorithm follows a Greedy approach, since, in the case of puzzle solving, identifying local optimum - merges between groups that are part of the ground truth, usually leads to the global optimum - the ground truth. At the same time, an exhaustive approach faces computational difficulties.

In the first stage, the puzzle is simplified by performing those merges for which the corresponding edges have distances small enough to be considered adjacent in the ground truth. In the second stage, merges are executed based on a voting system. The third stage involves identifying the groups that best fit into the remaining empty spaces, and if the solution does not converge into a single group, the remaining groups are disassembled and merged with the main group.

3.2.1 Stage I: Merging Groups Based on Edge Pairs with Low Matching Distance

Each piece initially starts as an individual group and a search is performed over all edge pairs, sorted in ascending order by the normalized color distance. For each pair, it is checked whether merging the edges E_a and E_m satisfies a specific individual condition. If both this condition and the conditions listed in 3.1 are met, one of the groups is dismantled, and the other is updated to reflect the result of merging the two groups.

The individual condition *Condition I* used in the initial phase of this stage is:

$$d_{\text{color}}^{\text{in}}(E_a, E_m) \cdot d_{\text{color}}^{\text{reciprocal}}(E_a, E_m) < th_1. \quad (3.6)$$

The threshold th_1 is experimentally determined to be 0.6 . The motivation behind this heuristic is that, at the beginning of the solving process, we aim to identify as many edge pairs as possible that are adjacent in the ground truth, with the guarantee that no incorrect merges are made.

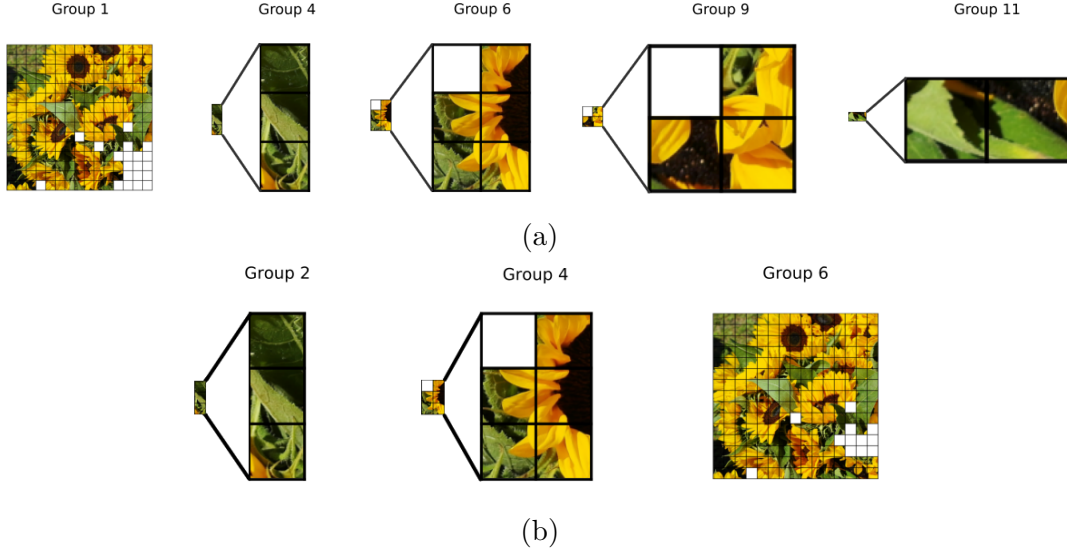


Figure 3.4: **Execution of Stage I**, example for a 15×15 puzzle. (a) *Result after applying Condition I.* (b) *Result after applying Condition II.*

This distance rejects any edge pair for which labeling is not straightforward (there are many pixel-level color differences greater than 0.3 or the pair E_a, E_m does not have strong reciprocity).

Although this filtering reduces the number of merges performed in this stage, the final algorithm is faster thanks to this strategy. This is because, based on the assumption that the configurations of the newly formed groups are part of the ground truth, there is no need to identify errors and undo groupings, but only to execute optimal merges. As shown in Figure 3.4a, most of the merges necessary to solve the puzzle are performed using the distance computed in Equation 3.6.

After all edge pairs that satisfy the previous condition are used to perform merges, the same strategy is applied again with a different condition, called *Condition II*:

$$\mathbb{E}[d_{\text{color}}^{\text{normalized}}(E_a, E_m)] < th_2, \quad \forall E_a \in G_a, E_m \in G_m \quad (3.7)$$

where E_a and E_m are the edges that become adjacent as a result of merging the groups they belong to, and th_2 is equal to 0.02 . This metric identifies new edge pairs that lead to merges, where the probability that they are adjacent in the ground truth is high.

By imposing a mean threshold of 0.02 , at least half of the new matches will have a distance less than or equal to 0.01 , a value that provides strong certainty regarding their presence in the ground truth. As a result, this condition allows for merging edge pairs that are indeed adjacent in the ground truth but have a distance that is too high to be caught by the first condition.

3.2.2 Stage II: The Voting System

In this stage, each edge E_a on the boundary of the formed groups, belonging to group G_a , searches for another edge E_m that is likewise on the boundary of the formed groups but does not belong to the same group, such that the distance between E_a and E_m is minimal. A vote has the form:

$$v_x = (G_a, G_m, \nabla_x, \nabla_y, \nabla_\theta),$$

where $\nabla_x, \nabla_y, \nabla_\theta$ denote the translation and rotation to apply to group G_m in its merge with G_a .

The condition used in Stage II is:

$$\begin{aligned} \mathbb{E}[d_{\text{color}}^{\text{normalized}}(E_a, E_m)] &< th_3, \\ \forall E_a \in G_a, E_m \in G_m, E_a, E_m \text{ adjacent.} \end{aligned} \quad (3.8)$$

where th_3 starts at 0.024 and is progressively increased by a factor of 1.2 once all merges that satisfy the previous threshold have been performed. The voting stage ends when th_3 reaches 1.0 . At every iteration, all votes that satisfy the condition (3.8) are counted by their unique key v_x , and the key with the most votes triggers the corresponding merge of groups G_a and G_m .

This strategy maximizes the available context: the merge executed is always the one with the highest vote count v_x among the keys whose edge distances are smallest. Consequently, the decision is influenced by the number of voters, giving priority to groups with many pieces, although the voter count is not the sole criterion. An approach based only on the number of votes would tend to merge edges with uniformly low-contrast colors (e.g., clear sky), merges that are often not found in the ground truth.

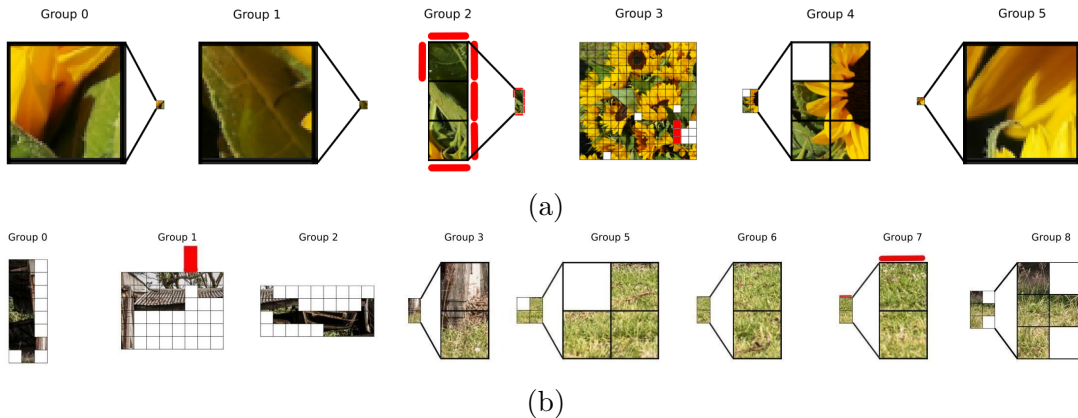


Figure 3.5: **Stage II**, examples for puzzles with 15×15 pieces. (a) *Example of a group merge present in the final solution.* (b) *Example of a group merge not present in the final solution.*

Figure 3.5a shows a merge between two groups of pieces in Stage II where Group 2 has 6 edges (marked in red) voting to merge with Group 3, using a translation of $(12, 10)$ and a rotation of 0° , placing Group 2 on the red-highlighted positions of Group 3. Each of these 6 edges individually had, among all possible votes, the smallest distance to the corresponding edges of the indicated merge. The votes were validated as feasible and their mean distance satisfied the variable threshold. Figure 3.5b presents a merge from the voting stage of another puzzle that is not found in the final solution. The edge from Group 7, marked with a red line, votes to merge with Group 1 using a translation of $(5, -3)$ and a rotation of 180° . In the set of votes whose mean lies in the range $[0.34, 0.40]$, only single-vote candidates exist, among which this merge has the lowest score.

3.2.3 Stage III: Merging the Remaining Pieces to the Main Group

For each unoccupied position x, y of the formed groups, the number of *neighboring* positions (up, down, left, right) that are already occupied is counted. The number of neighbors determines the constraint level of the position x, y . These unoccupied positions are then filled in descending order of their constraint level, such that the average of all new edge matches is minimized, using the *normalized color distance*.

There are puzzles where the algorithm does not succeed in reaching a single group of pieces, even after iterating through all edge pairs in the order of their matching distances, without applying any threshold. This phenomenon occurs because the remaining groups are geometrically incompatible. Next, only the group with the highest number of pieces is left unchanged, and all the other groups are disassembled. Pairings are then executed using the same strategy and the same criterion, the *normalized color distance*, with the important difference that the pieces from disassembled groups are only allowed to be matched to the main group, and not to each other.

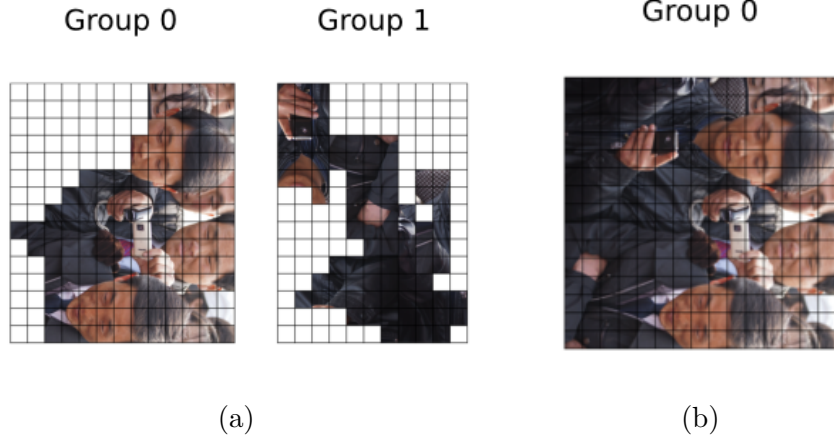


Figure 3.6: **Stage III**, example of a puzzle with 15×15 pieces. (a) Result after executing *Stage II*. (b) Result after executing *Stage III*.

Figure 3.6 illustrates a case resolved in Stage III, where two groups remain, and the second group contains piece merges that are not part of the ground truth, as the matching is difficult due to lack of detail. This group is disassembled and the correct placement is found for each individual piece. As a result, the final solution matches the ground truth.

In most cases, the pieces that were not merged to the main group during the previous stages are those whose edge pairings are difficult to determine accurately. Stage III is primarily intended to minimize the number of merges not present in the ground truth. If more than one group remains after the previous stages, there certainly exist incorrect pairings, as the remaining group shapes are incompatible for merging.

Figure 3.7a shows an intermediate puzzle state during Stage I, where incorrect pairings exist in Group 6 and Group 9. Figure 3.7b shows the result after completing Stage I, in which Group 3 contains multiple mismatches compared to the ground truth. After filling the gaps in Stage III, the puzzle state shown on the left in Figure 3.7c still contains errors in Group 1 compared to the ground truth. This is the result of a “domino effect” caused by the first incorrect pairing. Although Stage III reduces the number of mismatched pairings, it cannot eliminate them completely.

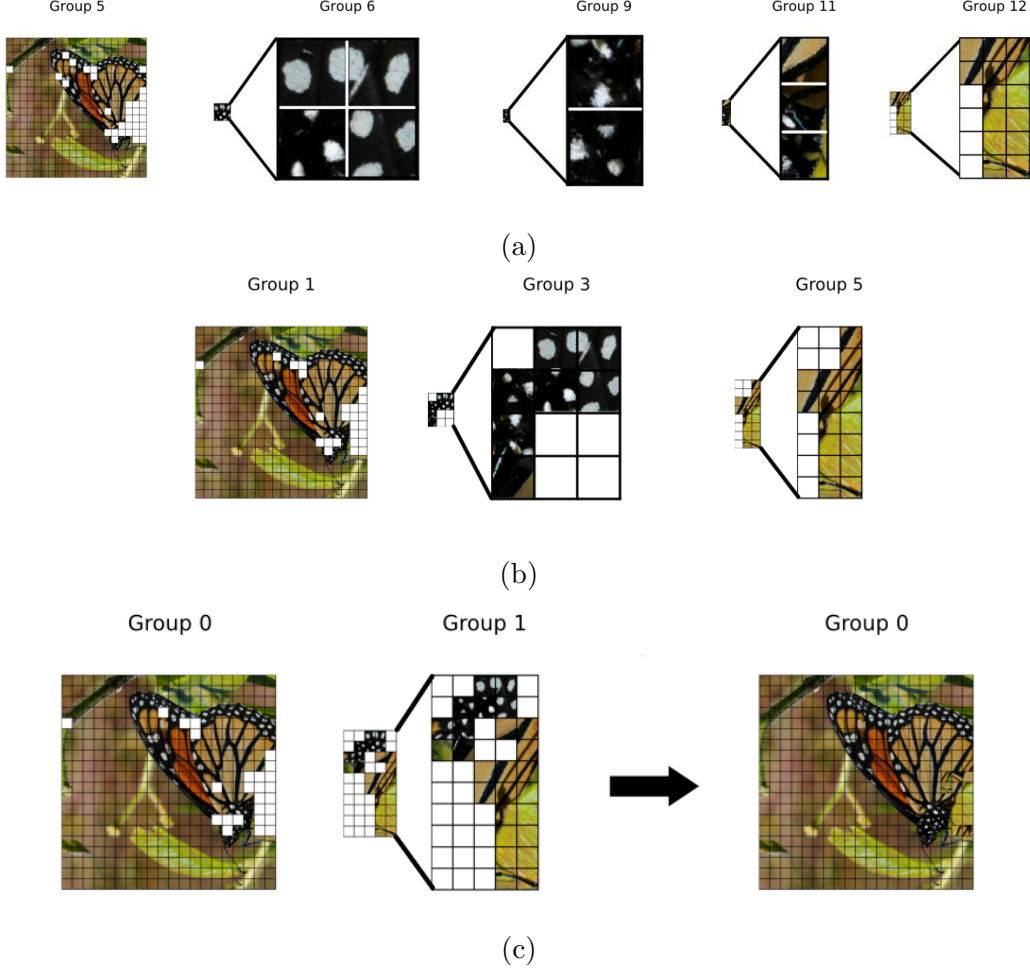


Figure 3.7: **Example of puzzle solving stages**, 15×15 pieces. (a) *Intermediate solving stage from Stage I.* (b) *Puzzle result after Stage I.* (c) *Gap filling (left), group disassembly and joining to the main group after executing Stage III (right).*

3.3 Results

The performance of each stage is measured based on the percentage of edge pairs that are adjacent in the ground truth at the end of that particular stage. This performance is evaluated based on the reconstruction of 120 natural images, with varying resolutions between 800×800 and 1400×1400 pixels, extracted from the *DIV2K* dataset. These images were divided into square pieces and used as input data in random order and rotation, forming puzzles of sizes 10×10 , 15×15 , and 20×20 .

Table 3.1 presents the percentages corresponding to each stage, and Figure 3.8 illustrates the distribution of performance percentages for each stage of the algorithm, for puzzles with 15×15 pieces.

Table 3.1: Performance of each solving stage of the algorithm

Puzzle Size	Stage I Condition I	Stage I Condition II	Stage II	Stage III
10×10	68%	78%	94%	92%
15×15	74%	81%	92%	85%
20×20	78%	82%	89%	82%

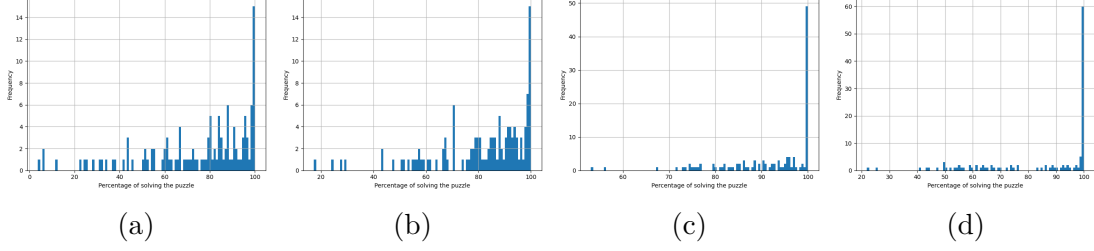


Figure 3.8: **Distribution of the percentage of edge pairs adjacent according to the final solution**, for 15×15 piece puzzles. (a) *Stage I, Condition I*. (b) *Stage I, Condition II*. (c) *Stage II*. (d) *Stage III*.

Most of the edge pairs that are present in the ground truth occur in Stage I, where over 80% of all adjacent edge pairs are present to the ground truth. In Stage II, an average of 91% of all adjacent edge pairs match the ground truth. Stage III reconstructs the puzzle so that it fits the final dimensions of the ground truth, thus, the overall reconstruction performance is not improved and even decreases to 86%, with results varying from case to case.

Once the puzzle is completed, with all pieces positioned, the performance of the solving algorithm can be evaluated using a *distance* metric, which sums the Euclidean distances between the misplaced pieces and their ideal positions. Although this work does not provide implementations aimed at minimizing such a metric, it allows for a comparison between the proposed algorithm and human solving, where incorrectly positioned pieces tend to be close to their correct location in the final solution. Naturally, as the puzzle size increases, these distances also increase, since pieces placed incorrectly tend to be located towards the edges of the puzzle. Table 3.2 presents the *mean distance* values obtained for puzzles of size 10×10 , 15×15 , and 20×20 .

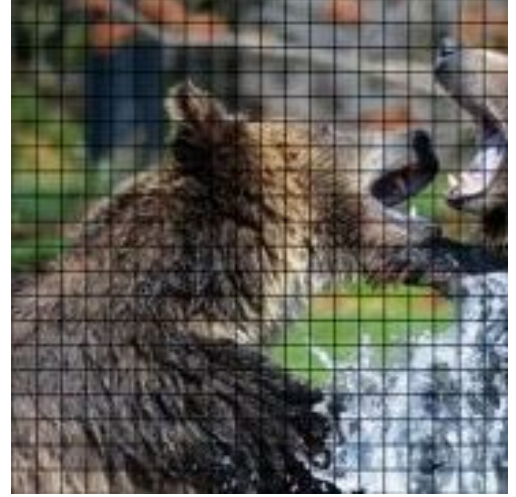
Table 3.2: Distance

Puzzle Size	Distance
10×10	53
15×15	346
20×20	826

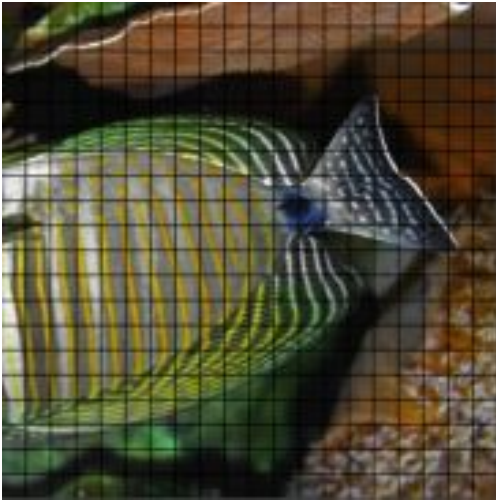
Most of the puzzles are correctly solved, with a distance of 0. At the same time,



(a)



(b)



(c)



(d)



(e)



(f)

Figure 3.9: **Examples of solved 20×20 puzzles.** (a) *Distance* = 0. (b) *Distance* = 0. (c) *Distance* = 0. (d) *Distance* = 0. (e) *Distance* = 1572. (f) *Distance* = 2580.

the more difficult puzzles tend to have significantly bigger distances, as a “domino effect” occurs: a small number of incorrect edge adjacent pieces causes the misplacement of many other pieces.

Figure 3.9 presents several examples of solved 20×20 puzzles, which are representative of the algorithm’s performance.

Conclusions

This work aimed to develop a range of distance metrics designed to discriminate between pairs of edges whose pieces are adjacent in the final puzzle solution and those that are not, alongside a Greedy algorithm for solving puzzles containing between 10×10 and 20×20 pieces. Throughout the development process, foundational concepts from the field of computer vision were applied, including smoothing filters and color-based distance measures.

The project involved exploring various methods and approaches, requiring iterative testing, refinement, and integration of components. The results demonstrate the applicability of a deterministic technique for puzzle solving, in which the decision-making process is transparent and interpretable, unlike the widely used neural network-based approaches, which operate as black boxes and rely on difficult to justify features learned from large amounts of data; therefore they are often difficult to justify. At the same time, the study illustrates the significant complexity imposed by heuristic methods, as well as their inherent limitations.

This project opens several directions for future development, such as improving the discriminative metrics or refining the decision-making component by implementing a decision tree module. Moreover, by extending the types of puzzles being solved, such as puzzles with distinct or irregular piece shapes, the work gains practical applicability in fields such as image restoration.

The current implementation achieves an average accuracy of 92% for 10×10 puzzles, 85% for 15×15 , and 82% for 20×20 puzzles, with the correct orientation of pieces being unknown. Solving square-piece puzzles without knowing their orientation remains an active research topic, with current efforts focused on leveraging neural networks. Prior to 2010, the best results using Greedy algorithms achieved successful assembly of up to 432 pieces with known orientations; today, state-of-the-art methods have reached performance levels of over 9,000 pieces with unknown orientation.

Bibliography

- [1] Naif Alajlan. “Solving Square Jigsaw Puzzles Using Dynamic Programming and the Hungarian Procedure.” In: *American Journal of Applied Sciences* (2009). URL: <https://thescipub.com/pdf/ajassp.2009.1941.1947.pdf>.
- [2] Taeg Sang Cho, Shai Avidan, and William T. Freeman. “A Probabilistic Image Jigsaw Puzzle Solver.” In: (2010). URL: <https://people.csail.mit.edu/billf/papers/JigsawSolverCVPR2010.pdf>.
- [3] Andrew C. Gallagher. “Jigsaw Puzzles with Pieces of Unknown Orientation.” In: (2012). URL: https://projet.liris.cnrs.fr/imagine/pub/proceedings/CVPR2012/data/papers/049_01A-04.pdf.
- [4] Nadia Guerroui and Hamid Seridi. “Solving computational square jigsaw puzzles with a novel pairwise compatibility measure.” In: *Journal of King Saud University - Computer and Information Sciences* (2018). URL: <https://doaj.org/article/7c327a75964f43cf8e482f258b2b1d38>.
- [5] Vahan Huroyan, Gilad Lerman, and Hau-Tieng Wu. “Solving Jigsaw Puzzles by the Graph Connection Laplacian.” In: *SIAM Journal on Imaging Sciences* (2020). URL: <https://doi.org/10.1137/19M1290760>.
- [6] Kilho Son, James Hays, and David B. Cooper. “Solving Square Jigsaw Puzzles with Loop Constraints.” In: (2014). URL: https://faculty.cc.gatech.edu/~hays/papers/puzzle_eccv14.pdf.